

Part 1: Synthesized Dataset Generation Strategy

Approach Overview:

The pipeline generates 3000 synthetic pharmacokinetic plots by learning parameter distributions from 150 real examples, then sampling from these distributions to create new variants that match both structure and statistics. [line and function approximation]

Libraries: matplotlib (Agg backend), NumPy, SciPy, PIL, JSON

Implementation Details:

- Checked if plots follow a common shape

I first inspected the 150 images to see if there was a strong shared curve pattern (like a clean PK family).

Result: the plots were very noisy and did not share one consistent pattern, so I didn't rely on a single fixed curve type.

- Matched error bar size distribution

I extracted these label fields from the 150 dataset JSONs: topBarPixelDistance, bottomBarPixelDistance, deviationPixelDistance. Then I computed realistic min/max ranges and saved them into: dataset_error_bar_ranges.json. During synthetic generation, the pipeline loads these learned ranges and uses them as the allowed pixel-distance range for new error bars. This ensures generated error bars fall into the same pixel-size range as the real dataset.

- Matched “error bar present vs absent” ratio

The pipeline uses:

- error_bar_probability = 0.7

Meaning ~70% of generated plots include error bars, matching the rough frequency observed in the 150 examples.

- Pixel-accurate error bar rendering

To ensure the error bar distances are correct in pixels (not just data units): Sample a target error size in pixels from the learned range. Then convert pixel size into a fraction of figure height, Convert that into a data-space error magnitude using y_range and image_height, Render error bars with matplotlib, Measure the final rendered result and write back the label fields:

topBarPixelDistance, bottomBarPixelDistance, deviationPixelDistance. This keeps the labels consistent with what appears in the image.

- Tried to match curve parameter ranges

I also added a learning step where the pipeline tries curve fitting from dataset points to learn ranges for exponential decay parameters:

Model used: $y = a \cdot e^{(-kx)} + by$

The learned parameter ranges are saved in: dataset_param_ranges.json. During generation, some curves sample (a, b, k) from these learned ranges, otherwise defaults are used.

Quality assurance:

By learning key distributions from the 150 real labels (min/max ranges for top/bottom/deviation pixel distances and the error-bar presence ratio) and then verifying the generated set matches these via quick sanity checks (histograms/summary stats for error-bar sizes, number of points/lines, image sizes, and presence ratio). Annotation accuracy by making error bars pixel-consistent: I sample a target pixel distance, render with matplotlib, then measure the rendered geometry and write the final topBarPixelDistance, bottomBarPixelDistance, and deviationPixelDistance from the actual output, so labels reflect what is truly drawn, not just what was intended.

Scalability

- To generate 3000+ images reliably without crashes or slowdowns, the pipeline uses basic scalability safeguards: it generates images in batches while keeping memory low by immediately saving outputs to disk and calling **close()** on each matplotlib figure (instead of storing arrays in RAM).
- For speed, it can run in parallel using multiprocessing/joblib where each worker produces a unique index range and writes to unique filenames to avoid collisions. It also supports resuming by skipping any sample whose PNG/JSON already exists.
- Finally, it runs quick quality checks after generation, verifying image/label counts match and that key distributions (error-bar size histograms and error-bar presence ratio) look consistent with the source dataset.
- Memory management: Agg backend, explicit gc.collect() every 50 images, batch processing (6×500 instead of 1×3000)
- Speed: 3000 plots in ~10 minutes
- Storage: 157MB total

Part 2: Error Bar Detection Strategy

Approach Overview

I formulated error-bar detection as a **multi-output regression** problem. For each data point (x, y) on a plot, the model predicts three pixel distances: topBarPixelDistance, bottomBarPixelDistance, deviationPixelDistance. I trained the regressor on ~3000 synthetic **labeled images** and evaluated it on the 150 real provided images. [Feature extraction, XGBoost]

Model Choice

1) CLIP (openai/clip-vit-base-patch32) as feature extractor

- Uses a pre-trained vision encoder to produce a **512-dim global image embedding**
- Chosen because it is robust and quick to run (≈ 0.1 s/image in my setup)

2) XGBoost (MultiOutputRegressor) for prediction

- Works well with **mixed feature types** (dense embeddings + numeric coordinates)
- Trains quickly on a few thousand samples (≈ 5 min for my run)
- Limitation: it does **not learn spatial hierarchies** like CNNs (important for local structures like error bars)

Training Setup

- **Inputs:** (CLIP_embedding + normalized point features)
- **Targets:** [topBarPixelDistance, bottomBarPixelDistance, deviationPixelDistance]
- **XGBoost hyperparameters (default):**
 - n_estimators = 300
 - max_depth = 6
 - learning_rate = 0.05
- **Metric:** MAE (Mean Absolute Error) during regression training

Testing / Evaluation Pipeline

1. From the 150 real images' JSON, I created a standardized input.json format containing the provided points.
2. I stored the real labels in Error_bar_groundTruth/.
3. Using the trained model, I predicted error bar distances for those same 150 images and saved outputs to Error_bar_prediction/.

4. I measured accuracy by counting a prediction as correct if all predicted distances matched ground truth within ± 5 pixels (tolerance = 5px).

Results

- Accuracy (± 5 px tolerance): 2.29%

Edge Cases Observed

- Overlapping error bars: use local crops around each point (patch-based features)

Root Cause Analysis (why accuracy is low)

- **Small effective training signal:** even though I used ~3000 synthetic images, the model still sees a limited variety of realistic error-bar styles compared to the 150 real plots. The real dataset has noise/plot styles that synthetic data did not fully capture.
- **Feature limitation:** global CLIP embedding is not designed to capture small local geometry like thin error bars
- **Data quality issues:** more overlapping bars, outliers, and noisy plots reduce learnability